Bioinformatics
&
Algorithms for Bioinformatics
2025/2026

Searching Similar Sequences in Databases
BLAST (Basic Local Alignment Search Tool)

Miriam Santos
[d**CC**] @ Faculty of Sciences University of Porto

Outline

- **Searching Similar Sequences in Databases**
  - **BLAST Algorithm**
    - Motivation
    - Basic Local Alignment Search Tool *(Overview)*
    - Implementing Simple BLAST
    - Heuristics for BLAST *(Seeding and Extending)*
    - BLAST Revisited (Deeper Dive)
    - Interpreting the Significance of the Alignments
    - BLAST Programs and Variants
    - Using BLAST through NCBI and BioPython

Needleman-Wunsch

		Y									
	ϵ	T	A	T	G	T	C	A	T	G	C
ϵ	0	-7	-14	-21	-28	-35	-42	-49	-56	-63	-70
T	-7	1	-6	-13	-20	-27	-34	-41	-48	-55	-62
A	-14	-6	2	-5	-12	-19	-26	-33	-40	-47	-54
C	-21	-13	-5	1	-6	-13	-18	-25	-32	-39	-46
X G	-28	-20	-12	-6	2	-5	-12	-19	-26	-31	-38
T	-35	-27	-19	-11	-5	3	-4	-11	-18	-25	-32
C	-42	-34	-26	-18	-12	-4	4	-3	-10	-17	-24
A	-49	-41	-33	-25	-19	-11	-3	5	-2	-9	-16
G	-56	-48	-40	-32	-24	-18	-10	-2	2	-1	-8
C	-63	-55	-47	-39	-31	-25	-17	-9	-3	-1	0

$s(a, b)$

	A	C	G	T	-
A	1	-3	-1	-3	-7
C	-3	1	-3	-1	-7
G	-1	-3	1	-3	-7
T	-3	-1	-3	1	-7
-	-7	-7	-7	-7	

Smith-Waterman Algorithm

Backtrace: (a) start from *maximal* cell, (b) stop upon reaching cell with score = 0

	ϵ	T	A	T	A	T	G	C	G	G	C	G	T	T	T
ϵ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
G	0	0	0	0	0	0	2	0	2	2	0	2	0	0	0
G	0	0	0	0	0	0	2	0	2	4	0	2	0	0	0
T	0	2	0	2	0	2	0	0	0	0	0	0	4	2	2
A	0	0	4	0	4	0	0	0	0	0	0	0	0	0	0
T	0	2	0	6	0	6	0	0	0	0	0	0	2	2	2
G	0	0	0	0	2	0	8	2	2	2	0	2	0	0	0
C	0	0	0	0	0	0	2	10	4	0	4	0	0	0	0
T	0	2	0	2	0	2	0	4	6	0	0	0	2	2	2
G	0	0	0	0	0	0	4	0	6	8	2	2	0	0	0
G	0	0	0	0	0	0	2	0	2	8	4	4	0	0	0
C	0	0	0	0	0	0	0	4	0	2	10	4	0	0	0
G	0	0	0	0	0	0	2	0	6	2	4	12	6	0	0
C	0	0	0	0	0	0	0	4	0	[illegible]					
T	0	2	0	2	0	2	0	0	0	[illegible]					
A	0	0	4	0	4	0	0	0	0	[illegible]					

```
x : GGTATGCTGGCGCTA
        |||||  ||||
y : TATATGC-GGCGTTT
```

$s(a,b)$

	A	C	G	T	-
A	2	-4	-4	-4	-6
C	-4	2	-4	-4	-6
G	-4	-4	2	-4	-6
T	-4	-4	-4	2	-6
-	-6	-6	-6	-6	

Searching Similar Sequences in Databases

In many situations in biological research, we come across DNA or protein sequences for which we do not have any information!

- How do investigators search through, organize, and make sense of massive amounts of data?
- How can they identify the functions of new genes?
- Is it possible to estimate the evolutionary relationship between genes or proteins by examining their nucleotide or aminoacid sequences?

Inferring from similar sequences is a way to address these questions, where we try to identify sequences that are similar to the target (query) sequence, which are already annotated.

Searching Similar Sequences in Databases

We need to scan databases and compare our query sequence against all sequences in the database, seeking to find the most similar ones!

- Can we use the algorithms we explored before?
- All we need is to run a pairwise sequence alignment between the query and all sequences in the database.
- This involves selecting the type of alignment (local or global), the gap penalty model, the substitution matrix (or match/mismatch scores).
- Those will affect the final results, i.e., the "similar sequences" found.

Searching Similar Sequences in Databases

This function takes a query, a list of sequences, a substitution matrix, and a gap penalty, and runs the Smith-Waterman algorithm to find the best local alignment of the query with a sequence in the database.

```
def align_query (query, ls, sm, g):
    bestScore = -1
    bestSeq = None
    bestAl = None
    for seq in ls:
        al = smith_Waterman(query, seq, sm, g)
        if al[2] > bestScore:
            bestScore = al[2]
            bestSeq = seq
            bestAl = al
    bestAlin = recover_align_local(bestAl[0], bestAl[1], query,
   bestSeq)
    return bestAlin, bestScore
```

https://github.com/miguelfrocha/BioinformaticsAlgorithmsBook/blob/master/Chapter6/align_ch6.py#L116

BLAST

Dynamic programming algorithms are not enough when we need to run them a large number of times (possibly millions!)

- One of the most popular solutions for this problem is **BLAST** (Basic Local Alignment Search Tool). BLAST has a **heuristic nature** that can work up to 100 times faster than the dynamic programming algorithms.

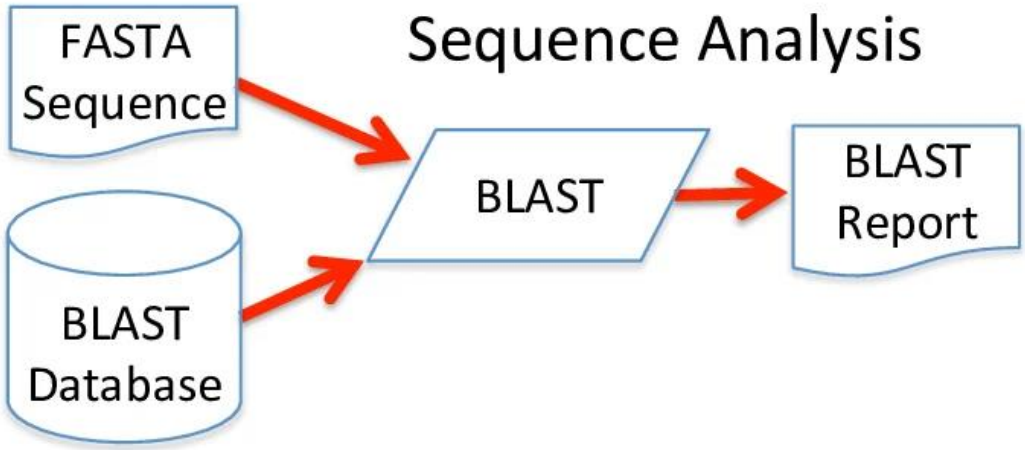
- However, at the cost of accuracy: **we lose the exact nature** of the algorithm, i.e, BLAST **does not guarantee to find the best possible** solution (alignment).

```
CAGTCAGAGCTCAGC   GAATCGGCGTACTGG
    ||||||        | | | | | | | |
GGACCAGAGCATTAA   GGACCAGAGCATTAG
```

BLAST: Basic Local Alignment Search Tool

- Powerful algorithm to compare biological sequences (proteins, DNA/RNA), offers a faster alternative to traditional methods.

- Its primary purpose is to compare a **query** sequence against a **database** of sequences, identifying those with a similarity above a certain **threshold**.



BLAST Overview

BLAST generally follows these key steps:

1. **Word Hits Generation:** The query sequence is divided into k-tuples, which are compared to pre-computed k-tuples from the database sequences.

2. **Seeding:** The matching k-tuples, known as "seeds", are used as anchor points for the alignment procedure. The list of k-tuples is expanded to include any k-tuples that match with an alignment score above a threshold ***T***.

3. **Extension:** BLAST extends these seeds in both directions, creating larger alignments. This extension continues until the alignment drops below a threshold, preventing poor alignments from extending. These extended matches are known as **High-Scoring Segment Pairs (HSPs)**.

BLAST Overview

4. **Alignment Scoring:** Each extended alignment is scored based on the similarity between the aligned regions.

5. **Statistical Evaluation:** BLAST calculates the statistical significance of each alignment, indicating how many alignments of the same quality would be found by chance in a database of a given size.

6. **Output:** The algorithm lists all alignments above a threshold score, providing details such as alignment positions, scores, and statistical significance.

Implementing Simple BLAST

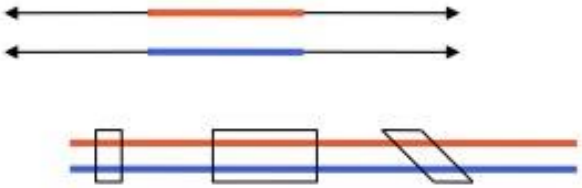
BLAST – no magic involved

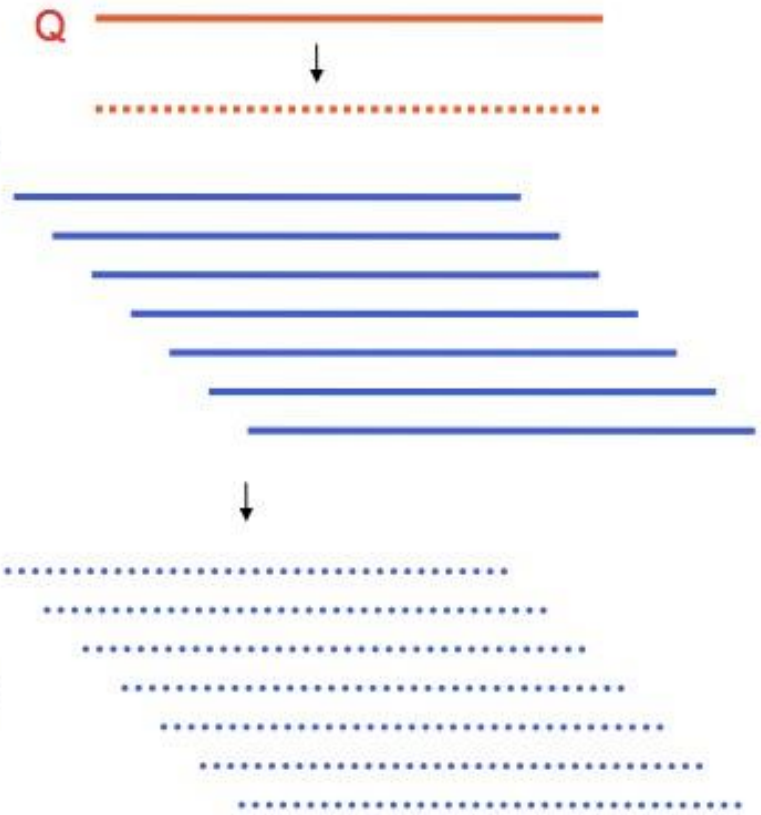
Concept:

1. Break query sequence (Q) and database into short 'words' of length W and create hash table

2. Seek matches between fragments – exceeding score T. Results in collection of high-scoring pairs (HSP).

3. Extend these matches as far as possible, until you get minus scores





Åsa Björklund credit

Implementing Simple BLAST

- **myblast.py**: A simplified version of BLAST.
- We will not use substitutions matrices.
- We will consider a score of 1 for a match and 0 for mismatch.
- We will not consider gaps.
- Only perfect hits are considered, i.e., the threshold $\boldsymbol{T}$ is equal to $\boldsymbol{w}$ (word length).

https://github.com/miguelfrocha/BioinformaticsAlgorithmsBook/blob/master/Chapter7/myblast_ch7.py

Implementing Simple BLAST

- ***Step 1: Create a "database"***
  - The database is a list of sequences (strings) which a read from a text file:

```
def read_database (filename):
    f = open (filename)
    db = []
    for line in f:
        db.append(line.rstrip())
    f.close()
    return db
```

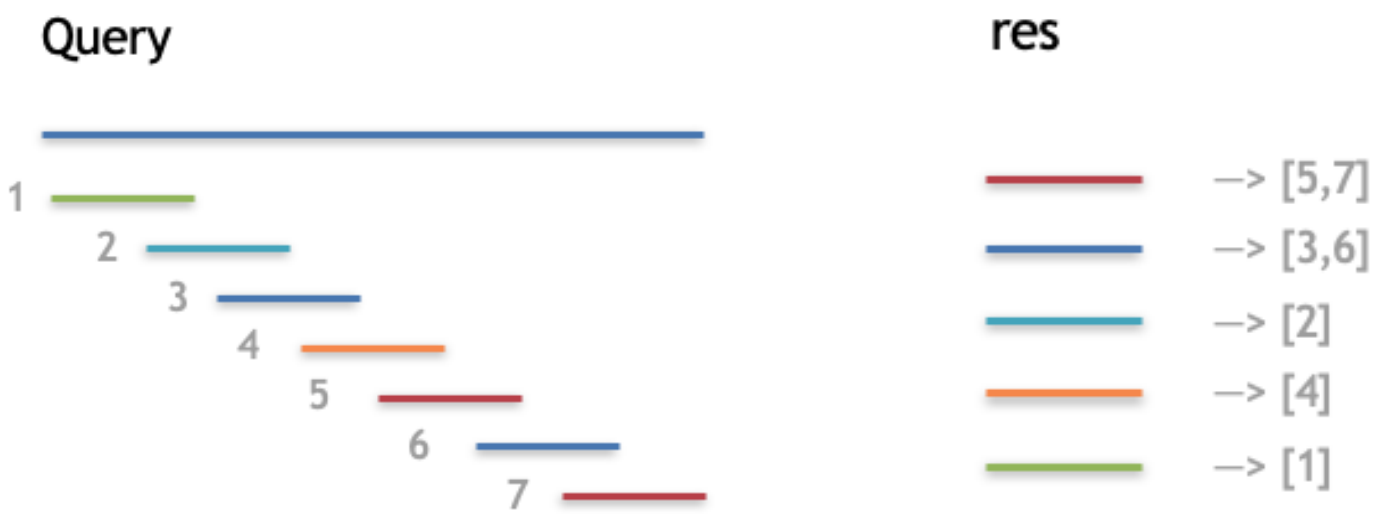
Implementing Simple BLAST

- ***Step 2: Create a map of words***
  - Find all words of size w and the positions in the query where they occur (*hashing*). Save it as a Python dictionary (words are keys, values are lists of positions where the words occur).

```
def build_map (query, w):
    res = {}
    for i in range(len(query)-w+1):
        subseq = query[i:i+w]
        if subseq in res:
            res[subseq].append(i)
        else:
            res[subseq] = [i]
    return res
```

Implementing Simple BLAST


```
def build_map (query, w):
    res = {}
    for i in range(len(query)-w+1):
        subseq = query[i:i+w]
        if subseq in res:
            res[subseq].append(i)
        else:
            res[subseq] = [i]
    return res
```



Implementing Simple BLAST

- ***Step 3: Find matching words between the query and the sequence***
  - Given a sequence (from db), we find all matches with the query. The result is a list of hits (tuples): (idx_query, idx_sequence). We use the map rather than the query itself.

```
def get_hits (seq, m, w):
    res = []  # list of tuples
    for i in range(len(seq)-w+1):
        subseq = seq[i:i+w]
        if subseq in m:
            l = m[subseq]
            for ind in l:
                res.append( (ind,i) )
    return res
```

map

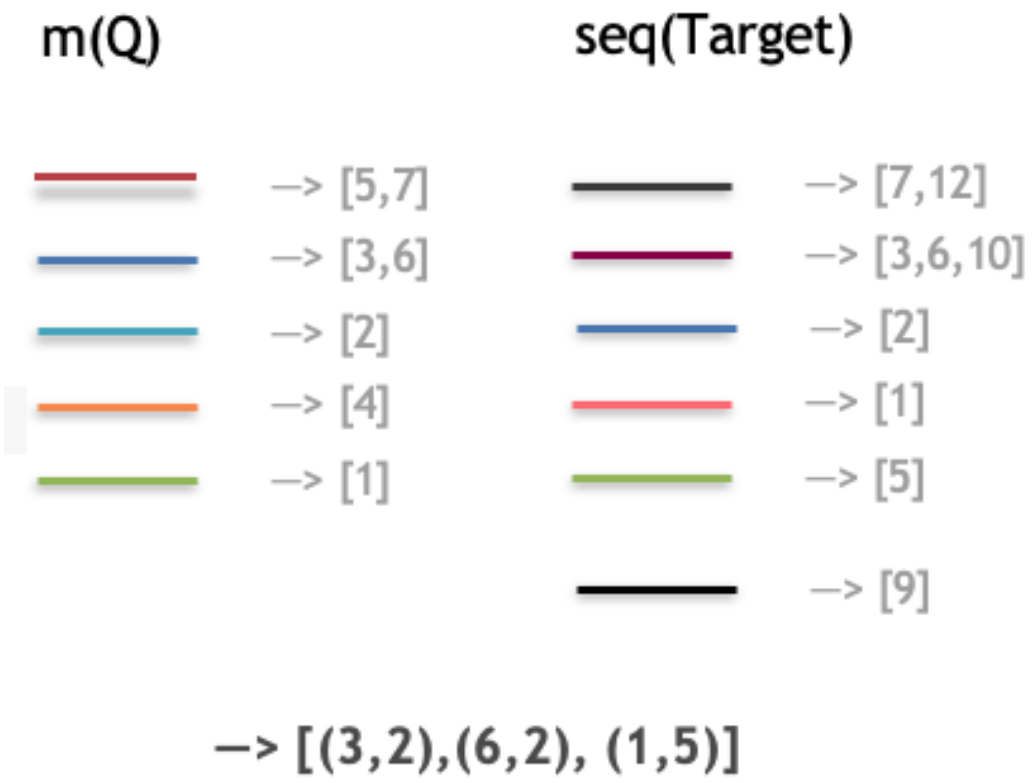
positions of word in the query

query

sequence

Implementing Simple BLAST

- ***Step 3: Find matching words between the query and the sequence***
  - Given a sequence (from db), we find all matches with the query. The result is a list of hits (tuples): (idx_query, idx_sequence). We use the map rather than the query itself.



Implementing Simple BLAST

- ***Step 4: Extend hits***
  - Hits will be extended in both directions (left and right) to see if it become a "good enough" alignment.
  - **What is "good enough"?** While the contribution to the increase in the score is larger or equal to half of the positions in the extension.
  - Meaning, **at least 50% of the position must match**, or we stop.

(ind, i) from get_hits

```
def extends_hit (seq, hit, query, w):
    stq, sts = hit[0], hit[1]
    ## move forward
    matfw = 0
    k=0
    bestk = 0
    while 2*matfw >= k and stq+w+k < len(query) and sts+w+k < len(seq):
        if query[stq+w+k] == seq[sts+w+k]:
            matfw+=1
            bestk = k+1
        k += 1
    size = w + bestk
```

starting points from **q**uery, **s**equence

matches when extended right direction

current extension legth

best extension legth

Implementing Simple BLAST


```
def extends_hit (seq, hit, query, w):
    stq, sts = hit[0], hit[1]
    ## move forward
    matfw = 0
    k=0
    bestk = 0
    while 2*matfw >= k and stq+w+k < len(query) and sts+w+k < len(seq):
        if query[stq+w+k] == seq[sts+w+k]:
            matfw+=1
            bestk = k+1
        k += 1
    size = w + bestk
    ## move backwards
    k = 0
    matbw = 0
    bestk = 0
    while 2*matbw >= k and stq > k and sts > k:
        if query[stq-k-1] == seq[sts-k-1]:
            matbw+=1
            bestk = k+1
        k+=1
    size += bestk

    return (stq-bestk, sts-bestk, size, w+matfw+matbw)
```

start position in the db sequence

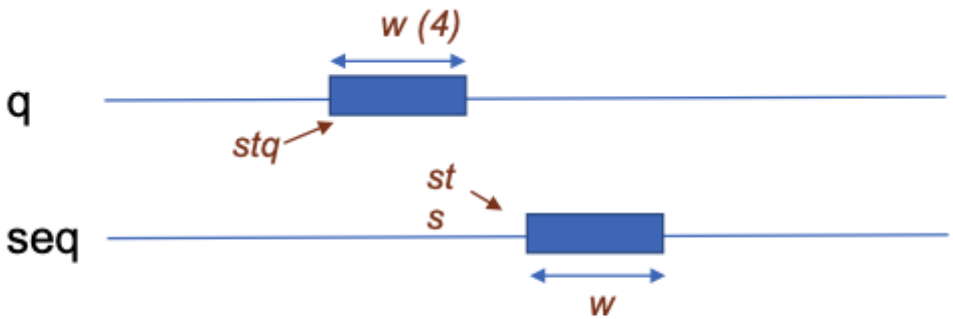
total number of matches (score)

new start of the alignment in the query after extending left

total length of the alignment

Implementing Simple BLAST

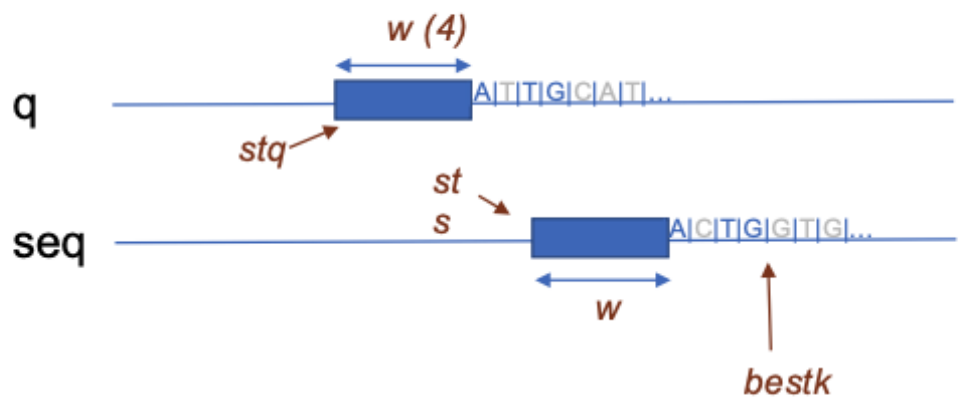
The result is a tuple: (starting index of alignment in on the query; starting index of alignment in on the sequence; size of the alignment, number of matching characters (score))



start_align_query | *stq*
start_align_seq | *sts*
size | *w*
score | *w*

k = 0; matfw = 0; bestk = 0;

move forward



2*matfw >= k	Test	matfw	bestk	k
-	-	0	0	0
0 >=0; yes	A == A	1	1	1
2>=1; yes	T == C	1	1	2
2>=2; yes	T == T	2	3	3
4>=3; yes	G == G	3	4	4
6 >= 4; yes	C == G	3	4	5
6 >= 5; yes	A == T	3	4	6
6 >= 6; yes	T == G	3	4	7
6 >= 7; no	-	-	-	-

Return:
bestk = 4
size | *w = 4 + 4 = 8*
score | *w + matfw = 4 +3 = 7*

Implementing Simple BLAST

- ***Step 5: Find best alignment (pick best extended hit)***
  - Identify all hits of size *w*, extend those hits, and select the one with the best overall score (number of matches). The result is a tuple of the same format as the last function.

```
def hit_best_score(seq, query, m, w):
    hits = get_hits(seq, m, w)
    bestScore = -1.0
    best = ()
    for h in hits:
        ext = extends_hit(seq, h, query, w)
        score = ext[3]
        if score > bestScore or (score==bestScore and ext[2] < best[2]):
            bestScore = score
            best = ext
    return best
```

total number of matches (score)

override best

If two alignments have the same score, choose the **shorter one (denser/better alignment)**

Implementing Simple BLAST

- ***Step 6: Apply the previous to all sequences in the database***
  - We find the best alignment with each sequence, and then the best overall alignment with a given sequence. The result is a tuple similar to those described before, adding in the last position the index of the sequence with the best alignment (idx in the db).

```
def best_alignment (db, query, w):
    m = build_map(query, w)
    bestScore = -1.0
    res = (0,0,0,0,0)  ⟶  res = bestSeq[0], bestSeq[1], bestSeq[2], bestSeq[3], k
    for k in range(0,len(db)):
        bestSeq = hit_best_score(db[k], query, m, w)
        if bestSeq != ():
            score = bestSeq[3]
            if score > bestScore or (score==bestScore and bestSeq[2] < res[2]):
                bestScore = score
                res = bestSeq[0], bestSeq[1], bestSeq[2], bestSeq[3], k
    if bestScore < 0: return ()
    else: return res
```

Simplified vs Real BLAST

1. **Scoring System**
   - Simplified version used match/mismatch = 1/0, no penalties
   - Real BLAST uses substitution matrices + gap penalties (biological meaningful)

2. **Word Generation (Seeds)**
   - Simplified version takes exact substrings of size *w*
   - Real BLAST considers similar words (above a threshold ***T* → *HSSP threshold***)

3. **Hit Detection**
   - Simplified version considers exact matches only and direct dictionary lookup
   - Real BLAST allows near matches and uses optimized indexing and lookup tables

4. **Extension**
   - Simplified version extends while >=50% matches
   - Real BLAST uses score-based extension (stops when score is < **extension threshold**)

5. **Best Alignment**
   - Simplified version returns one best alignment
   - Real BLAST returns many alignments (HSP), ranking them statistically

6. **Statistical Significance**
   - Simplified version returns a score that has no probabilistic meaning
   - Real BLAST computes the number of matches by chance (E-value)

Simplified vs Real BLAST

The BLAST Algorithm

Pre-process the database

Step 1: Construct the list of the k-mers whose ungapped alignment score with a k-mer of the input is within the HSSP score threshold.

Step 2: Find all the matches between k-mers in this list and the database.

Step 3: Seed the matches and extend.

Step 4: Evaluate the statistical significance.

https://youtu.be/wfi_KimrNQM?t=62

Evaluating Statistical Significance

- ***When we find HSPs, it is important to define their "significance"***
  - *e.g. we obtain a score of 54*

LIAWHCMPNAAA → query protein sequence

HIAACLMNAPWA	WNHACAPMIAAL	AAAWPNAIMCHL	AAMPLAANCIWH	APINAAHCMALW
APWIHAMLACNA	AACAPNIAWMLH	CALWHIPAAMNA	AMNPACWIHLAA	IPMCNAHLWAAA
AAIWPHLCMAAN	CWIALNAMPAAH	HCAIAALPAWNM	HAAALWAIMPNC	HAAMWAACILPN
NAAALAPMWCIH	NACALWMIPAHA	MACNHWALPIAA	ANAAMHAPWILC	LACHMANIAPWA
AWHAIMALCNAP	IAHAWCNMAPLA	PMAAAWLHNIAC	AWHIALACMNPA	AHALMNAPIACW
ACLAIMHPWNAA	APAANMCLHWIA	MCIAWPLAHANA	PCNAHAWIAMLA	AHALAWMNCIAP
MPCAWNHAIAAL	LAPWAMANAIHC	HIACPANAALWM	PIMLNAWAHAAC	HAWMANAALCIP
HPAALAMANWIC	NAWLPMAHCAIA	HMIPNAALCAAW	ANWAHMPACLIA	CAAANMIAPHWL
WANIHLMCAAPA	HANPCAWIALMA	ACANMAWILHPA	AALAAHPCMNWI	AMHWNAALACIP
AHMAACIWNPAL	AAAIPLAWNMHC	AAPAAWIMNHLC	AIAHNAMLPWAC	WHAIANLAACMP
PCLWMNAAAHIA	MLHWAPNAAACI	MNAAAWCLIPAH	IMLWNPCHAAAA	LAAAPWANMCIH
MAAAIHLWNAPC	HIAMWCLPNAAA	PAAMALICANWH	AMICLHPWAANA	IMAHAWNCLPAA
AIPACMALHWNA	PACAHAWLMAIN	ALAHPCAWNIMA	CAWHLPIAAAMN	ICLPHAAWNAAM
AWNAMLPAICHA	HMLPAAIWAACN	ACIAWNHAAMLP	WIANLPMAACHA	AHWPAIMALCNA
ANMAHAICLPAW	IANMHCAAALWP	WNAHAACAPILM	MAWAAIPNCLAH	CHAAAWMIPNAL
PANILACWAMHA	MAAPLWNHIAAC	NLHAAPWAMAIC	AHWAPCNMAALI	LAAHPNMWICAA
LWINAHPAMCAA	AMAHWPCNALAI	ANLCMPWHAAAI	LAIMCNAAHAPW	CALAWANAIMHP
MAWILCAHPAAN	WNAIMAPLAACH	LAAMHIAAPNWC	IWANMCHLAAAP	AHAPNCMAIWAL

all possible random combinations of those aminoacids

Evaluating Statistical Significance

- ***When we find HSPs, it is important to define their "significance"***

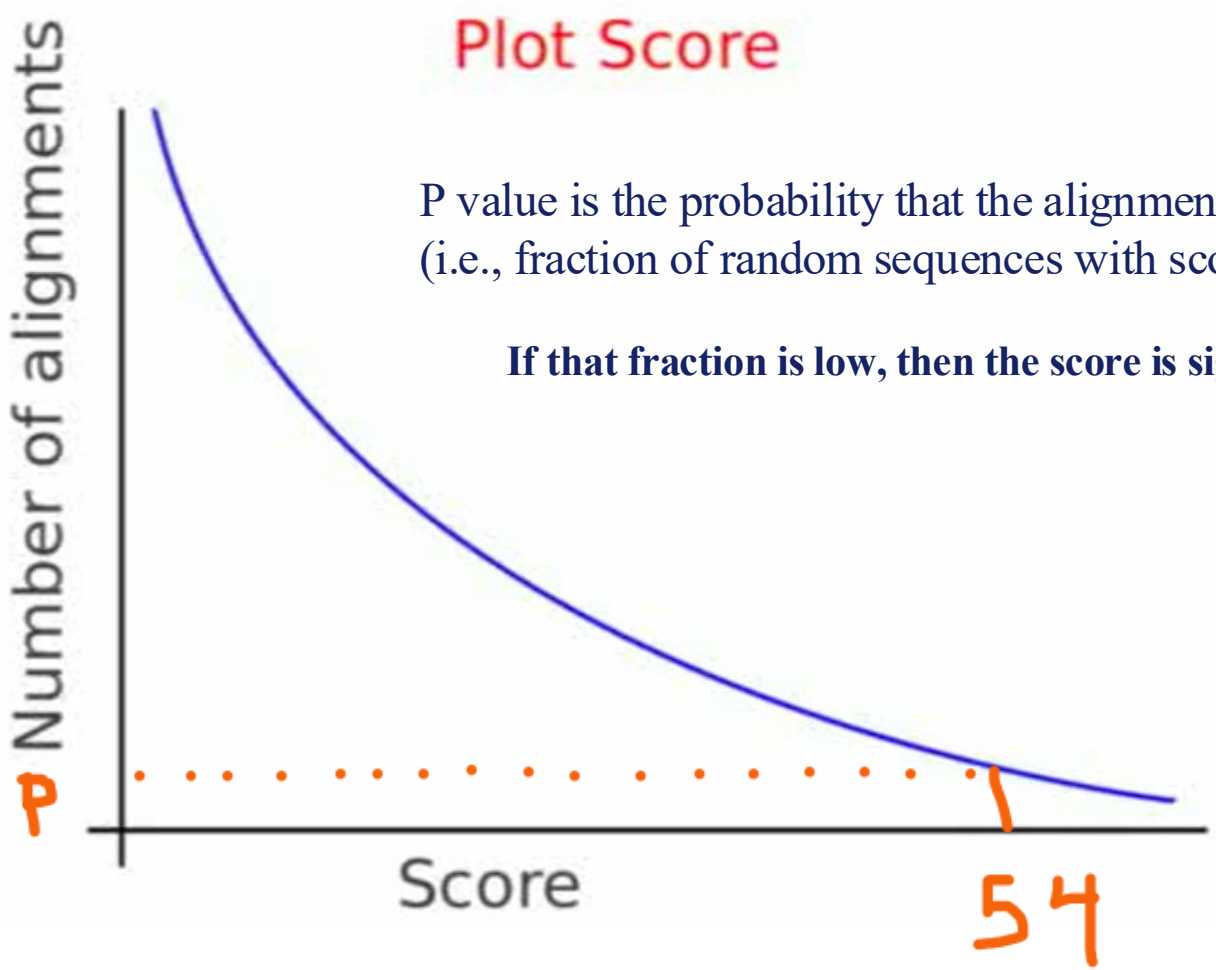
IIAWHCAMPNDDA → database protein sequence

HIAACLMNAPWA	45.5	AAAWPNAIMCHL	44.5	APINAAHCMALW	36.7
APWIHAMLACNA	13.7	CALWHIPAAMNA	12.2	IPMCNAHLWAAA	44.8
AAIWPHLCMAAN	49.7	HCAIAALPAWNM	24.6	HAAMWAACILPN	35.8
NAAALAPMWCIH	44.4	MACNHWALPIAA	2.8	LACHMANIAPWA	51.6
AWHAIMALCNAP	12.1	PMAAAWLHNIAC	25.4	AHALMNAPIACW	0.3
ACLAIMHPWNAA	37.5	MCIAWPLAHANA	16.9	AHALAWMNCIAP	3.8
MPCAWNHAIAAL	51.7	HIACPANAALWM	25.8	HAWMANAALCIP	0.3
HPAALAMANWIC	27.8	HMIPNAALCAAW	43.4	CAAANMIAPHWL	44
WANIHLMCAAPA	53.8	ACANMAWILHPA	50.6	AMHWNAALACIP	30.8
AHMAACIWNPAL	26.5	AAPAAWIMNHLC	39	WHAIANLAACMP	27.8
PCLWMNAAAHIA	44.3	MNAAAWCLIPAH	45.1	LAAAPWANMCIH	0.2
MAAAIHLWNAPC	25.8	PAAMALICANWH	27.7	IMAHAWNCLPAA	29.8
AIPACMALHWNA	43.8	ALAHPCAWNIMA	43.2	ICLPHAAWNAAM	29.1
AWNAMLPAICHA	51.1	ACIAWNHAAMLP	37.9	AHWPAIMALCNA	41.9
ANMAHAICLPAW	32.7	WNAHAACAPILM	34.6	CHAAAWMIPNAL	48.2
PANILACWAMHA	20.1	NLHAAPWAMAIC	45.9	LAAHPNMWICAA	7.2
LWINAHPAMCAA	8.2	ANLCMPWHAAAI	27.9	CALAWANAIMHP	47.3
MAWILCAHPAAN	23.9	LAAMHIAAPNWC	7.6	AHAPNCMAIWAL	19.6

alignment scores of random combinations

Evaluating Statistical Significance

- ***When we find HSPs, it is important to define their "significance"***



P value is the probability that the alignment is random (i.e., fraction of random sequences with score >= 54)

If that fraction is low, then the score is significant!

E-values

- ***BLAST is more complex***
  - Longer sequences are more likely to find HSPs
  - Longer databases are more likely to find HSPs

- ***BLAST converts P-values to E-values***
  - E = length of database (m) * length sequence (n) * probability (P)

$$\mathrm{E} = mnP$$

- Probability is converted into Expectation

What is the expected frequency we would find this alignment in the database, just at random?

E-Values

- The number of sequences with HSP score of S is:

$$\mathrm{E} = kmne^{-\lambda S}$$

- ***k*** and ***λ*** are scaling factors (BLAST parameters, derived from earlier large-scale calibrations)
- ***m*** is the length of the database
- ***n*** is the length of the query sequence
- ***S*** is the raw HSP score (e.g., 54)
- ***S*** stays the same, but ***E*** grows with ***m*** (depends on the size of the database -- all databases grow!)
- Larger databases result in higher E-values (weaker statistical significance) for the same level of sequence similarity!

E-values

E-value	Interpretation
$< 1 \times 10^{-100}$	Identical
1×10^{-50} to 1×10^{-100}	Nearly Identical
1×10^{-5} to 1×10^{-50}	Homologous
1×10^{-1} to 1×10^{-5}	Distantly Homologous
$> 1 \times 10^{-1}$	Maybe Random

Using BLAST through NCBI

National Library of Medicine
National Center for Biotechnology Information

Log in

BLAST ®

Home Recent Results Saved Strategies Help

Basic Local Alignment Search Tool

BLAST finds regions of similarity between biological sequences. The program compares nucleotide or protein sequences to sequence databases and calculates the statistical significance. Learn more

The Basic Local Alignment Search Tool (BLAST) finds regions of local similarity between sequences. The program compares nucleotide or protein sequences to sequence databases and calculates the statistical significance of matches. BLAST can be used to infer functional and evolutionary relationships between sequences as well as help identify members of gene families.

NEWS

Mon, 21 Jul 2025

Here are a few highlights in our latest BLAST+ release:

Download BLAST+ 2.17.0 now!

More BLAST news...

Web BLAST

Nucleotide BLAST
nucleotide ▶ nucleotide

blastx
translated nucleotide ▶ protein

tblastn
protein ▶ translated nucleotide

Protein BLAST
protein ▶ protein

BLAST Genomes

Enter organism common name, scientific name, or tax id | Search

Human Mouse Rat Microbes

https://blast.ncbi.nlm.nih.gov/Blast.cgi

Using BLAST through NCBI

National Library of Medicine
National Center for Biotechnology Information

Log in

BLAST ® » blastn suite

Home Recent Results Saved Strategies Help

blastn blastp blastx tblastn tblastx

Homo sapiens (human) Nucleotide BLAST

BLASTN programs search nucleotide databases using a nucleotide query. more...

Enter Query Sequence

Enter accession number(s), gi(s), or FASTA sequence(s) Clear

Query subrange

From

To

Or, upload file Choose File no file selected

Job Title

Enter a descriptive title for your BLAST search

Choose Search Set

Database Genome (GRCh38.p14 reference assembly RS_2024_08)

Exclude Optional Models (XM/XP)

Entrez® Query Optional

Enter an Entrez query to limit search

Program Selection

Optimize for
- Highly similar sequences (megablast)
- More dissimilar sequences (discontiguous megablast)
- Somewhat similar sequences (blastn)

Choose a BLAST algorithm

BLAST

Search database Genome (GRCh38.p14 reference assembly RS_2024_08) - Homo sapiens using Megablast (Optimize for highly similar sequences)

Show results in a new window

+ Algorithm parameters

Homo sapiens (human) Nucleotide BLAST

5 Types of BLAST

Program	Query	Database	Notes	Typical Uses
BLAST**n**	Nucleotide	Nucleotide	Nucleotide BLAST	DNA annotation; phylogenetics
BLAST**p**	Protein	Protein	Protein BLAST	Common regions between proteins
BLAST**x**	Nucleotide	Protein	Queries are translated into protein	Finding protein-coding genes
tBLAST**n**	Protein	Nucleotide	The database is translated into protein	Mapping a protein to genomic DNA
tBLAST**x**	Nucleotide	Nucleotide	Queries and database are translated intro protein	Searching for genes missed or not yet in protein databases

Which BLAST program should I use?

- BAST**n** and BLAST**p** are the most commonly used.

- Sometimes, another BLAST program may produce interesting hits.

  - If a nucleotide sequence is translated before the search (all possible 6 reading frames) it is more likely to find better and more accurate hits than a BLASTn search (protein sequences are evolutionarily more conserved than nucleotide sequences)

  - Also, proteins hits are likely to be annotated (you can directly see the protein function of the sequenced gene)

What you cannot get out of BLAST

- Don't expect BLAST to produce the best available alignment.
- BLAST is a heuristic method which does not guarantee the best results (BLAST may not find all the hits in the database).
- BLAST makes only local alignments (great but short hits in a database sequence may not be related to the query sequence!)
- Don't expect the default settings to provide the best result. Unfortunately, there is not a final answer on how to adjust the settings for your particular sequence…
- protein function of the sequenced gene)

Bioinformatics
&
Algorithms for Bioinformatics
2025/2026

Searching Similar Sequences in Databases
BLAST (Basic Local Alignment Search Tool)

Miriam Santos
[d**CC**] @ Faculty of Sciences University of Porto